

ACHILES

Platform Assurance

The quiet platform heartbeat. Privately checks whether XELOR is responding, preserves the result history and hands failed evidence to the people and agents responsible for incident coordination and repair.

Who this is for: product, engineering, operations and anyone who has to explain XELOR to somebody else.

What it covers: the work ACHILES reasons about, the rules it cannot break, the exact tools it can call, and what is honestly not built yet.

Truth standard: the repository as it stands on 8 August 2026.

ACHILES

What ACHILES is for

A role with a fixed tool list, not a general assistant

In one sentence

Privately checks whether XELOR is responding, preserves the result history and hands failed evidence to the people and agents responsible for incident coordination and repair.

3

business areas it speaks for

1

tools it can actually call

1

capability families allowed

0

agents it may delegate to

What reaches it

Hourly scheduler trigger · Authorised internal operator trigger · Configured private service endpoints · Tenant identity and access context

What it hands back

Healthy, degraded or unavailable status · Per-component pass/fail evidence and latency · Freshness warning after 90 minutes · A bounded hand-off to RELAY and the technical owner

Capability families it is allowed to touch

platform-health.

Ownership and authority are not the same thing

The areas above are where ACHILES is the right specialist to ask. The tool table on page 8 is the much smaller set it can invoke at runtime. Owning a business area does not grant runtime power over it — and for ONYX, the supervisor, the gap between the two is the widest in the system.

What sits behind ACHILES

Records, screens, workflow, controls and hand-offs

Private Platform Assurance		ACHILES module · platform-health
Purpose	Quietly checks whether XELOR's API, tenant database, event queue, public web application and declared AI runtime are responding, then keeps the evidence private.	
Core records	Append-only platform-health runs with tenant, trigger, overall status, component results, latency, start/completion time and total duration.	
User surface	Private status. APIs: /platform-health/overview, /platform-health/run and the secret-protected /internal/platform-health/run scheduler endpoint.	
End-to-end flow	Run deterministic probes with four-second timeouts; classify required and optional failures; save the tenant-isolated result; mark evidence stale after 90 minutes; hand failed evidence to RELAY and the technical owner.	
Enforced controls	Internal-only permissions, tenant RLS, append-only storage, one concurrent sweep, no model judgement, no ERP write, no service restart, no customer message and no action-dispatch capability.	
Inputs and outputs	ACHILES detects and records. RELAY coordinates the incident and customer update. HEXA, ONYX or the affected specialist diagnoses and repairs.	
Current boundary	The probes, scheduler paths, permissions, screen and durable history are live MVP capability. It is not a production observability stack, paging service, root-cause engine or autonomous remediation system.	
Primary code map	apps/web/src/modules/platform-health/manifest.ts · apps/api/src/modules/platform-health/	

How to read these module pages

“Live” means the records, service rules and user/API surface exist in this repository. It does not mean every surrounding enterprise process is complete. The boundary row names what remains illustrative, manual, simulate-driven or outside the MVP.

How ACHILES's modules connect

A module owns its records; the agent explains the combined evidence

Cross-module coordination

ACHILES detects and records only. RELAY owns incident severity, clock and customer updates. HEXA, ONYX or the affected specialist owns diagnosis and repair. A person keeps control of high-impact decisions.



How ACHILES's world actually runs

The business pipeline, not the agent plumbing

ACHILES is deliberately simpler than a general AI agent. It answers one operational question using explicit technical probes, records exactly what happened and stops before diagnosis or repair.

1

Wake privately

An internal interval or secret-protected external scheduler starts one sweep. A second overlapping sweep is refused.

2

Enter tenant context

Each active tenant is checked inside its own identity and row-level-security boundary.

3

Probe fixed components

Check API, PostgreSQL, Valkey and the configured web URL with explicit timeouts. Declare the AI runtime mode without inventing an external model result.

4

Classify honestly

A required failure means unavailable; an optional failure means degraded; an unconfigured optional endpoint is shown as not configured, never passed.

5

Preserve evidence

Insert one immutable run with component details, latencies, trigger and timestamps. The latest 24 are shown to authorised operators.

6

Hand off and stop

Failed evidence goes to RELAY and the relevant technical owner. ACHILES does not diagnose, restart, repair, message a customer or close an incident.

How much of this ACHILES can actually see

Of everything above, ACHILES can read exactly one thing directly — latest private status, freshness and append-only history. The rest of this pipeline is context for judging what it reads; it is not data the agent can reach. Where a mission needs more, it asks the specialist who owns it or it says the evidence is missing.

What the code will not let anyone do

Enforced by constraints and locks, not by wording

Evidence, not an AI guess

Platform state comes from deterministic probes with timeouts.
No language model decides whether XELOR is healthy.

Private by permission

Only xelor_admin, it_admin and demo_admin receive the view/run permissions; ordinary customer roles do not see the module.

Observation is not remediation

The allow-list contains one read capability and no agent.action.dispatch entry.

Missing is not healthy

An endpoint that was not configured is labelled not configured.
A check older than 90 minutes is labelled stale.

Every tenant remains separate

The scheduler enters explicit tenant context and the database applies row-level security before reading or saving evidence.

One failure does not hide the rest

A failed tenant check is counted as unavailable while the sweep continues across the other active tenants.

Why this page matters more than the agent pages

An agent is only as trustworthy as the system it reads. Every rule here holds whether the request came from a person, a screen, a seeder or ACHILES — which is precisely why an agent can be given a read tool without also being given a way to do damage.

Where these rules are kept

apps/api/src/modules/platform-health/

apps/web/src/modules/platform-health/

packages/db/src/schema/platform-health.ts

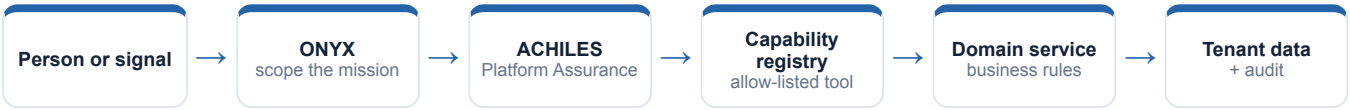
packages/db/migrations/0070_achiles_platform_health.sql

apps/api/src/agent-os/capability-registry.service.ts

docs/01-agent-os/05-achiles-platform-assurance.md

How ACHILES takes part in a mission

The engine controls order, parallelism and recovery



1 · Scope
ONYX turns the goal into a run against a frozen graph version, with a fixed tenant, user, step budget and timeout.

2 · Authorise
Two checks, both required: the tool must be on ACHILES's allow-list, and the signed-in person must independently hold the permission.

3 · Execute
A registered domain service does the work under the same rules a screen would face. There is no general SQL doorway.

4 · Preserve
Inputs, outputs, events and a checkpoint after every wave, so the run can be explained or resumed.

An agent can narrow authority, never widen it
ACHILES is a specialist and delegates to nobody. Because the permission check is repeated against the requesting person, a mission can never reach data that person could not have opened themselves.

Everything ACHILES can call

This table is the complete list — there is no other door

Capability	Mode	What comes back	Permission required	Needs approval
platform-health.status.read	Read	Latest private status, freshness and append-only history	platform_health.overview.read	No

Read · Analyse · Simulate

Return evidence or a reproducible view. Nothing in the business changes.

Draft

Produce reviewable content that is labelled a draft and can never present itself as an approved outcome.

Execute

The only side-effecting mode in the whole system, and the only one that requires an approved human gate above it.

Both locks must open

The agent's allow-list AND the person's permission. Either one failing is a refusal.

ACHILES cannot act in the business at all

There is no execute capability on this list, and ACHILES is not on the allow-list for the one that exists. The agent that coordinates every mission is the only one that cannot cause an effect — which is the point, not an oversight.

Where ACHILES actually appears

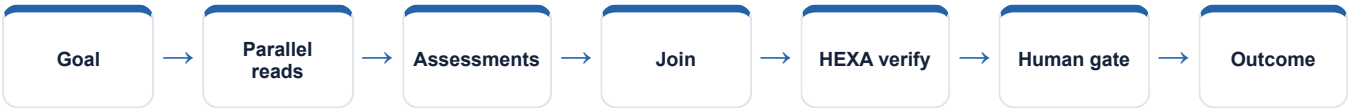
Node by node, from the registered graph definitions

Mission graph	What ACHILES does in it
Factory flow recovery	Preserves the boundary: reports XELOR health and never assesses or commands the robot
Cross-functional readiness	Not involved
Nine-agent operating review	Reads private platform status and assesses availability and evidence freshness
Controlled action mission	Reads and assesses platform health, then remains read-only after the human gate
Working Capital Review	Not involved
QMS & Audit Readiness	Not involved
Managed Service Assurance	Not a graph participant; supplies private detection evidence upstream of RELAY

Reading this honestly

“Not involved” is not a limitation, it is the design. A mission asks only the specialists whose evidence it needs, and a specialist cannot add itself to a graph at runtime. The graph is written down, versioned and content-hashed before the run starts.

The shape every mission shares



An authorised operator checks the demo before a meeting

Real records from the running demo,
not an illustration

The operator opens ACHILES → Private status and presses Run private check now. Ordinary customer roles cannot open this screen.

The API and tenant database pass. Valkey returns PONG. The configured public web page returns a healthy HTTP response. Each result carries measured latency.

ACHILES records one manual observation and the screen changes to XELOR is working. The new append-only row appears above earlier hourly observations.

If a component fails, the screen says degraded or needs attention. ACHILES records the evidence and stops; RELAY and the technical owner take over from there.

The sentence to say out loud

“ACHILES contributes evidence from its own area, with the source records behind it and its limits stated. It does not claim an action is done until the system has recorded and verified it.”

Fair questions to ask while watching

- Which records is this answer standing on?
- Which permission was checked, and against whom?
- Is this a recorded fact, a simulation, a draft, or a completed result?
- Would this step have waited for a person?
- What happens if the service restarts halfway through?

Live today, and deliberately not

The second column is the one worth reading

Working now • ACHILES as the ninth and final agent in the registry, map, catalogue and guided agent tour • Permission-gated Private Platform Assurance module and manual check • Hourly in-process scheduler for long-running deployments • Secret-protected scheduler endpoint and Vercel cron declaration • Real API, PostgreSQL, Valkey and web probes with bounded timeouts • Tenant-fenced append-only result table, freshness state and latest-24 history • Read-only Agent OS capability in both nine-agent mission graphs	Not built — stated plainly • No production telemetry aggregation, distributed tracing, paging or on-call integration • No root-cause diagnosis, predictive anomaly model or automatic remediation • No customer-facing status page or outbound notification channel • A simple endpoint response does not prove every business workflow is correct • Exact hourly execution depends on the selected host or external scheduler supporting that cadence
--	---

Identity boundary	Verified token → tenant and actor
Authority boundary	Agent allow-list AND the person's permission
Action boundary	Side effects need an approved ancestor node
Data boundary	Domain service over a tenant-fenced database
Evidence boundary	Append-only runs, events, checkpoints and audit

Gaps are listed because a guide that hides them fails the first hour of technical due diligence. Every item on the right is either a roadmap decision or a known defect, and both are recorded in the project's own capability-gap register.

ACHILES on one page

For explaining the agent to somebody new

Its job

Privately checks whether XELOR is responding, preserves the result history and hands failed evidence to the people and agents responsible for incident coordination and repair.

Speaks for

Private platform status, Hourly deterministic checks, Availability evidence history

Takes in

Hourly scheduler trigger · Authorised internal operator trigger · Configured private service endpoints · Tenant identity and access context

Gives back

Healthy, degraded or unavailable status · Per-component pass/fail evidence and latency · Freshness warning after 90 minutes · A bounded hand-off to RELAY and the technical owner

Can call

platform-health.status.read

Cannot do

No production telemetry aggregation, distributed tracing, paging or on-call integration · No root-cause diagnosis, predictive anomaly model or automatic remediation · No customer-facing status page or outbound notification channel

Words used on these pages

Agent	A named specialist with a fixed, allow-listed set of tools — not a process that can roam.
Capability	One registered operation with a declared mode, owner and required permission.
Mission graph	A versioned recipe: which steps run, which run together, where it waits for a person.
Checkpoint	A stored snapshot after each wave, used to explain a run and to resume it safely.
Governed work item	An approved, append-only assignment for a human team — not the business change itself.